Universidad Industrial de Santander Facultad de Ingenierías Fisicomecánicas Escuela de Ingeniería de Sistemas e Informática		
Asignatura: Estructuras de Datos y Análisis de Algoritmos	**Código:** 22955	**Grupo:** B1/C1
Docente: Laura Viviana Galvis Carreño	**Correo:** lavigal@uis.edu.co	

Taller 1. Introducción a las Estructuras de Datos y Recursividad

Febrero 26 de 2026

Instrucciones: El informe del taller debe subirse al moodle, antes del **10 de Marzo a las 8:00 pm**. Informe enviado después de la fecha y hora estipulada no será tenido en cuenta.

El informe debe incluir el código en Java programado en cada uno de los ejercicios cuando se indique y como se indique, así como los resultados de la ejecución en forma de captura de pantalla. Se debe crear **un archivo comprimido** con el nombre **"EDA_B1/C1_Taller1_*Nombre_Apellido*",** en caso de que se pida un proyecto y un **archivo PDF del informe (resultado de la plantilla de Latex/Overleaf) con el mismo nombre "EDA_B1/C1_Taller1_*Nombre_Apellido*"**.

---

1. SOBRE INTRODUCCIÓN A LAS ESTRUCTURAS DE DATOS

1.1. Manejo y comprensión de los principios de POO: Se quiere modelar el tipo de personas que hacen parte de la comunidad UIS. Particularmente, existe la clase genérica Persona de la cual descienden Estudiante y Empleado. A su vez, de la clase Estudiante descienden Estudiante_Pregrado y Estudiante_Posgrado. De esta última clase descienden Estudiante_Maestria y Estudiante_Doctorado. Por otra parte, de la clase Empleado descienden Profesor y Administrativo.

Todas las personas tendrán como mínimo los atributos: *nombre, apellido, edad*, y el método común: *saludar()*, que describe quién es, cuántos años tiene y qué rol cumple dentro de la Universidad. Siéntase libre de agregar más atributos y/o métodos.

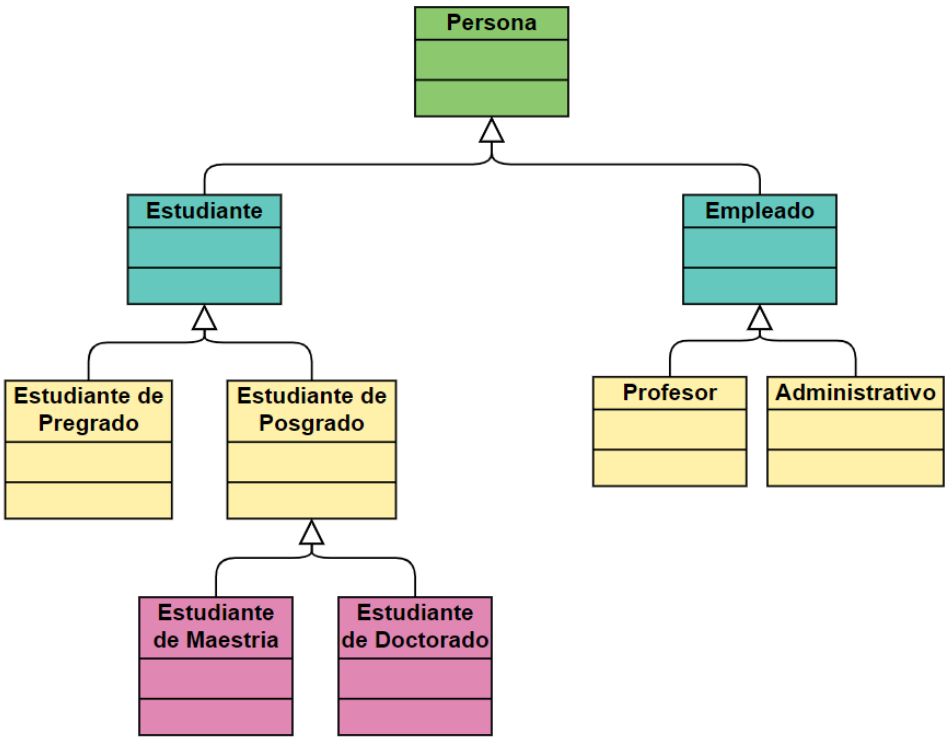
Entregables:

- Diagrama de clases UML, representando la herencia entre clases, el *modificador de acceso* de cada atributo y método, así como el *tipo de datos*. Como referencia, se adjunta una versión inicial (**incompleta**) del diagrama de clases esperado.

- Código programado en Java de cada una de las clases diseñadas incluyendo *getters/setters*. Incluir comentarios.

- Clase ***Main.java***, con método public static void main() que crea una instancia de cada una de las clases, e imprime en consola su forma de *saludar*. Adicionalmente, si el objeto pertenece a la clase Estudiante debe imprimir cuántos cursos está viendo, cuál es su promedio, en qué semestre va y qué programa académico está cursando. Si es empleado, debe imprimir su salario, unidad académica/administrativa (UAA) en la que trabaja y años que lleva trabajando en la UIS. Siéntase libre de agregar más información en cada caso (teniendo en cuenta su escalafón en la herencia).

- Captura de pantalla de la ejecución del código.



1.2. Crear una función que ordene un vector de tamaño fijo de manera ascendente o descendente usando la clase Arrays. La función debe recibir dos parámetros: el primer parámetro es el vector de tamaño fijo (de tipo numérico o String) y el segundo parámetro es un String que describe la forma en cómo se debe ordenar el vector de entrada ("ascendente" o "descendente"). Incluir código (*lstlisting*) y captura de ejecución en el informe.

```
public static int[] ordenarNumeros (int[] vector, String tipoOrden){
    // si tipoOrden es "ascendente"
```

```
3
4        // si tipoOrden es "descendente"
5       return vector;
6    }
```

```
1    public static String[] ordenarStrings (String[] vector, String tipoOrden){
2        // si tipoOrden es "ascendente"
3
4        // si tipoOrden es "descendente"
5
6       return vector;
7    }
```

1.3. Se busca diseñar un programa que reciba notas de estudiantes (enteros entre 1 y 100 puntos) para mostrar estadísticas de rendimiento. Se planea que el programa recibirá las notas de 300 estudiantes. Si el programa solo requiere imprimir las frecuencias por encima de 70 puntos. ¿Cuál sería la estructura de datos eficiente y necesaria para que el programa logre su objetivo? Justificar su respuesta y motivos de su selección.

a. Un Array con un tamaño de 300 int
b. Un Array con un tamaño de 100 int
c. Un Array con un tamaño de 30 int
d. Un Array dinámico de 370 int

1.4. ¿Cuáles de las siguientes declaraciones son verdaderas sobre la herencia en Java?

1. Los métodos **protected** son **final**
2. Los métodos **private** son **final**
3. No es posible **@Override** métodos **private**
4. Un miembro **protected** es accesible dentro del **package** donde se define y por las clases heredadas fuera del **package**

a. Solo 2 y 4
b. 1, 2 y 4
c. 1, 3 y 4
d. 2, 3 y 4

2. SOBRE RECURSIVIDAD

2.1. Reemplace `//Write your code here` en el siguiente código; es decir, implemente el método recursivo `encontrarSuma()`. Este método calcula la suma de los enteros positivos menores e iguales a **n**. Por ejemplo, `encontrarSuma(4)` debe imprimir 10. Reporte el código (*lstlisting*) y captura de pantalla de su ejecución.

```
public class Recursion {

    public static void main(String[] args) {
        System.out.println(encontrarSuma(8));
    }

    public static int encontrarSuma(int n){

        // Write your code here
    }
}
```

2.2. ¿Qué hace el siguiente método recursivo?. ¿En sus palabras, cuál es la recursión que resuelve? Explique.

a. a + (b - 1)

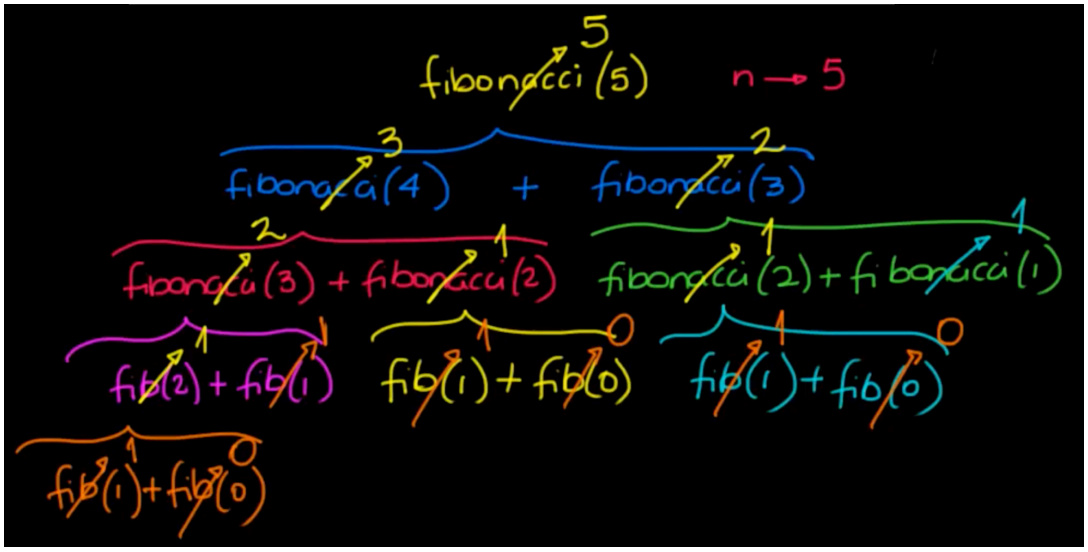
b. a * b

c. a + a * b

d. a^b

```
    static int hacer(int a, int b){
        if (b == 0){
            return 0;
        }
        else{
            return (a + hacer(a,b-1));
        }
    }
```

2.3. Fibonacci optimizado: Uno de los ejemplos que vimos en la clase sobre recursión, fue el método de calcular la secuencia de Fibonacci: 0, 1, 1, 2, 3, 5, … Discutimos que el método recursivo de Fibonacci tiene un problema cuando se trata de **eficiencia**, ya que si llamamos a fib(5), la función recursiva llama una vez a fib(4), dos veces a fib(3), tres veces a fib(2), cinco veces a fib(1) y tres veces a fib(0), como se muestra en la figura.



Pregunta: Diseñe y programe en Java una mejor estrategia recursiva para calcular el número de fibonacci. En particular, cree una función fibonacciOptimizado(), de tal manera que cuando se llame a fibonacciOptimizado(5), solo llame **una única vez** a la función fibonacciOptimizado(4), fibonacciOptimizado(3), fibonacciOptimizado(2), fibonacciOptimizado(1) y fibonacciOptimizado(0).

Incluir código (*lstlisting*) y captura de ejecución en el informe.